

# Module 8: Multidimensional Arrays

---

Sarah Foss

# Objectives

---

Identify and describe the structure and purpose of multidimensional arrays, including 2D, 3D, and higher dimensions.

---

Demonstrate the ability to declare and create two-dimensional arrays, specifying rows and columns to store tabular data.

---

Use specific row and column indices to access and modify elements within a two-dimensional array.

---

Apply nested loops to traverse and process data within rows and columns of multidimensional arrays.

# Multidimensional Arrays

A multidimensional array is like a collection of smaller arrays grouped together, or an array of arrays.

Imagine it as a grid of values, like cells in a spreadsheet.

Multidimensional arrays can help store and organize data that has rows and columns (and beyond!) – perfect for tables, game boards, and more.

# Two-Dimensional Arrays

A two-dimensional (2D) array organizes data in rows and columns, like a table or matrix.

|   | 0   | 1   | 2   |
|---|-----|-----|-----|
| 0 | 'O' | 'O' | 'X' |
| 1 | 'O' | 'X' | 'X' |
| 2 | 'X' | 'X' | 'O' |

Each cell is an element in the array!

Declaring syntax:

```
datatype[][] arrayName;
```

Example:

```
int[][] myNumbers;
```

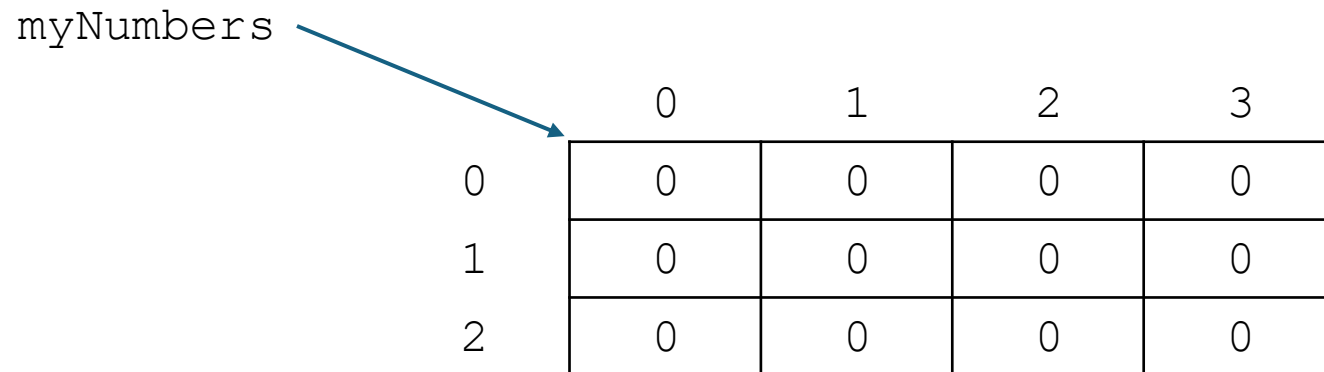
# Initializing a 2D Array

Initializing a 2D array with the **new** keyword:

```
arrayName = new datatype[rowSize][columnSize];
```

Example:

```
myNumbers = new int[3][4];
```



A diagram illustrating the initialization of a 2D array. The variable `myNumbers` is shown on the left, with a blue arrow pointing to the top-left cell of a 3x4 grid. The grid has rows indexed 0, 1, and 2, and columns indexed 0, 1, 2, and 3. Every cell in the grid contains the value 0.

|   | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 2 | 0 | 0 | 0 | 0 |

# Initializing a 2D Array with Values

You can create an 2D array and initialize it in one step:

```
int[][] matrix = {{1, 2, 3},  
                  {4, 5, 6},  
                  {7, 8, 9}};
```

matrix

|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 9 |

# Accessing Array Elements

Individual elements in a two-dimensional array are accessed with

`arrayName[row][column]`

where row and column refer to the position within the table.

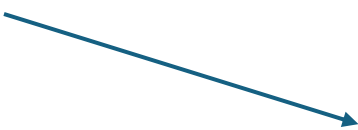
Example:

```
System.out.println(matrix[1][2]);
```

```
matrix[0][0] = 8;
```

```
matrix[2][2] = 0;
```

matrix



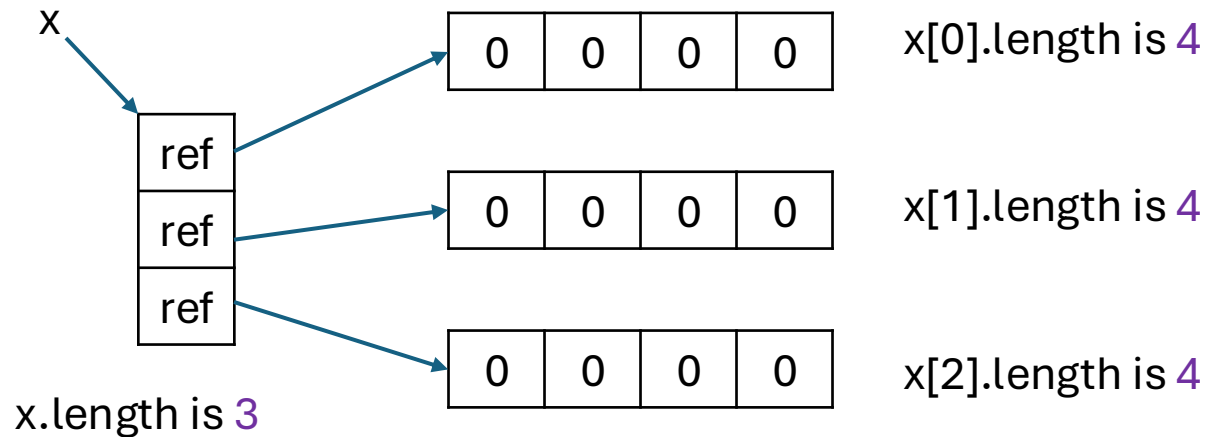
|   | 0 | 1 | 2 |
|---|---|---|---|
| 0 | 8 | 2 | 3 |
| 1 | 4 | 5 | 6 |
| 2 | 7 | 8 | 0 |

# Length

In arrays, the length property gives the total count of elements along a dimension.

For 2D arrays, array.length gives the number of rows, and array[row].length gives the columns in that row.

```
int[] x = new int[3][4];
```



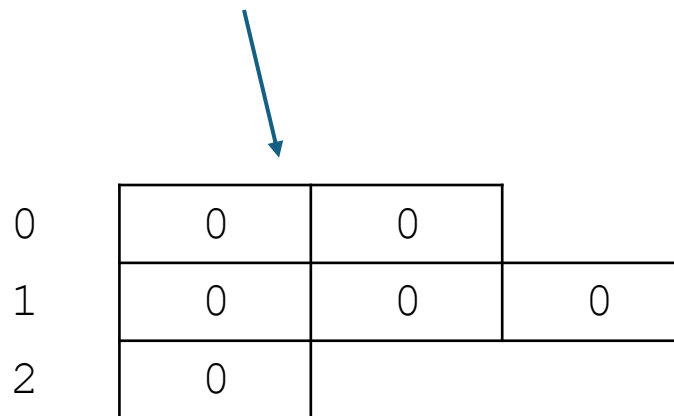


# Ragged Arrays

Ragged arrays have rows with different lengths.

```
int[][] raggedArray = new int[3][];  
raggedArray[0] = new int[2];  
raggedArray[1] = new int[3];  
raggedArray[2] = new int[1];
```

raggedArray

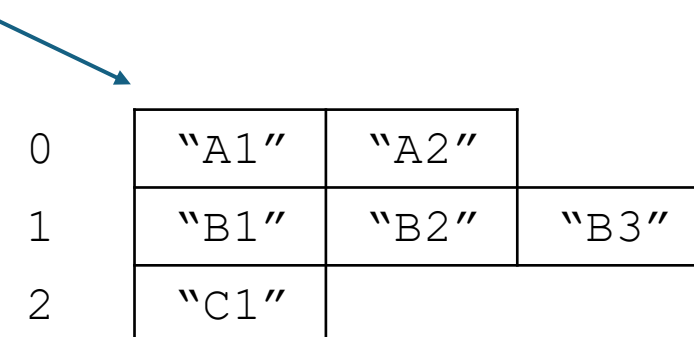


|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 |   |
| 1 | 0 | 0 | 0 |
| 2 | 0 |   |   |

raggedArray.length is 3  
raggedArray[0].length is 2  
raggedArray[1].length is 3  
raggedArray[2].length is 1

```
String[][] seats = {{ "A1", "A2"},  
                    { "B1", "B2", "B3"},  
                    { "C1" }};
```

seats



|   |      |      |      |
|---|------|------|------|
| 0 | "A1" | "A2" |      |
| 1 | "B1" | "B2" | "B3" |
| 2 | "C1" |      |      |

Here, each row represents a theater row with different seat counts.

# Looping Through a 2D Array

Use one loop for rows and another inside it for columns to go through each element:

```
for (int row = 0; row < matrix.length; row++) {  
    for (int col = 0; col < matrix[row].length; col++) {  
        System.out.printf("%3d", matrix[row][col]);  
    }  
    System.out.println();  
}
```

# Assigning With Input Values

```
Scanner input = new Scanner(System.in);

for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print("Enter value: ");
        matrix[row][col] = input.nextInt();
    }
}
```

# Assigning Random Values

```
for (int row = 0; row < matrix.length; row++) {  
    for (int col = 0; col < matrix[row].length; col++) {  
        matrix[row][col] = (int) (Math.random() * 100);  
    }  
}
```

# Summing All Elements

```
int total = 0;
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        total += matrix[row][col];
    }
}
```

# Summing Elements by Column

```
for (int col = 0; col < matrix[0].length; col++) {  
    int total = 0;  
    for (int row = 0; row < matrix.length; row++){  
        total += matrix[row][col];  
        System.out.println("Sum for column " + col + " is " + total);  
    }  
}
```

# Random Shuffling

```
for (int i = 0; i < matrix.length; i++) {  
    for (int j = 0; j < matrix[i].length; j++) {  
        int ranI = (int) (Math.random() * matrix.length);  
        int ranJ = (int) (Math.random() * matrix[i].length);  
  
        int temp = matrix[i][j];  
        matrix[i][j] = matrix[ranI][ranJ];  
        matrix[ranI][ranJ] = temp;  
    }  
}
```

# Exercise

Write a program to grade a multiple-choice exam.

Students' answer

|           | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-----------|---|---|---|---|---|---|---|---|---|---|
| Student 0 | A | B | A | C | C | D | E | E | A | D |
| Student 1 | D | B | A | B | C | A | E | E | A | D |
| Student 2 | E | D | D | A | C | B | E | E | A | D |
| Student 3 | C | B | A | E | D | C | E | E | A | D |
| Student 4 | A | B | D | C | C | D | E | E | A | D |
| Student 5 | B | B | E | C | C | D | E | E | A | D |
| Student 6 | B | B | A | C | C | D | E | E | A | D |
| Student 7 | E | B | E | C | C | D | E | E | A | D |

Key to the Questions:

|     | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-----|---|---|---|---|---|---|---|---|---|---|
| Key | D | B | D | C | C | D | A | E | A | D |



# Multi-dimensional Arrays

Sometimes, you'll need to work with data in multiple dimensions. You can create arrays with any number of dimensions  $n$ .

The same approach used to declare and create two-dimensional arrays can be applied to  $n$ -dimensional arrays for any  $n \geq 3$ .

Example:

```
String[][][] library = new String[2][4][3];
```

(or you could initialize it with values)

```
String[][][] library = {  
    {  
        {"Dune", "1984", "It"},  
        {"Emma", "Dracula", "Beowulf"},  
        {"Matilda", "Gatsby", "Beloved"},  
        {"Coraline", "Hatchet", "Inferno"}  
    },  
    {  
        {"Hamlet", "Odyssey", "Dorian Grey"},  
        {"Macbeth", "Watchmen", "Maus"},  
        {"Siddhartha", "Emma", "Preacher"},  
        {"Frankenstein", "Persuasion", "Congo"}  
    }  
};
```

*The first dimension represents the floor  
The second dimension represents shelf  
The third dimension represents books*

`library[1][3][0]` could represent a book on floor 1, shelf 2, book 0.

# Exercise: Calculating Total Scores

Create a program to calculate each student's total score.

Scores are stored in a three-dimensional array called scores:

- The first index represents each student.
- The second index represents each exam.
- The third index represents the exam part (multiple-choice or programming).

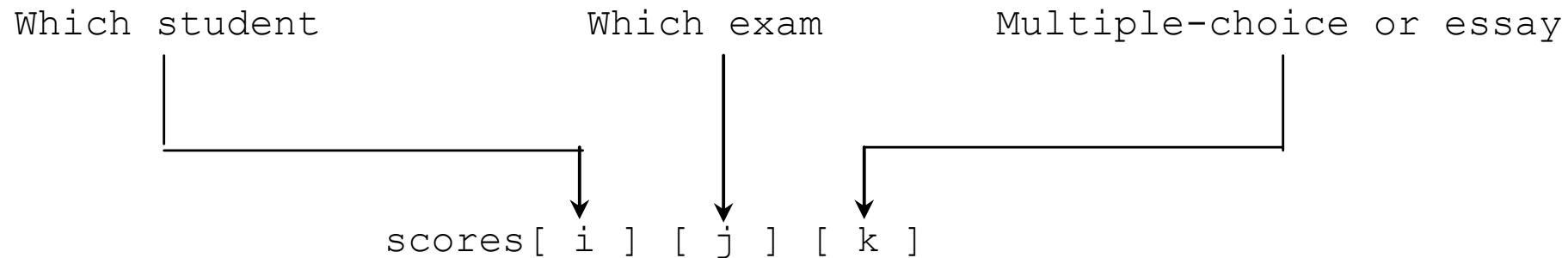
For example, there are 6 students, 5 exams, and each exam has two parts:

- `scores[i][j][0]` gives the multiple-choice score for student *i* on exam *j*.

The program should calculate and display each student's total score across all exams.

# Problem: Calculating Total Scores

```
double[][][] scores = {
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}
};
```



# Summary

---

- A multidimensional array is an array containing other arrays.
- Two-dimensional array arrays provide an effective way to store tabular data.
- To declare a two-dimensional array variable: `dataType[][] arrayName;`
- A two-dimensional array can be created with the syntax `arrayName = new dataType[rows][columns]`, where `rows` and `columns` specify the dimensions of the array.
- Individual elements in a two-dimensional array are accessed with `arrayName[row][column]` where `row` and `column` refer to the position within the table.
- Looping: Use nested for loops to iterate through rows and columns.
- 3D Arrays: Arrays with three indices, e.g., `int[][][]`.
- You can create arrays of arrays for higher dimensions. For instance, a 3D array can be declared with `dataType[][][] arrayName`, 4D array with `dataType[][][][] arrayName`, and so on.